

Netflix Recommendation System



- 201805051-Utku Enes Baki(Team Leader),
- 191805060-Semih Utku Polat,
- 191805061-Salih Can Aydoğdu,
- 201805044-Murat Tan

0. Introduction of data(Utku Enes(%50), Salih(%50))
1. Computer info (with code) IP address, name and configuration of our PC etc.(Semih Utku(%50), Murat(%50))
2. Starting the pyspark session, making the data workable(Salih Can(%33), Semih Utku(%33), Utku Enes(%33))
3. Recognizing the fact that data preprocessing must occur and taking steps(Salih Can(%50), Semih Utku(%50))
4. Some information about the dataset we have(Semih Utku(%50), Salih(%50))
5. The process of showing how many users are for each rating count(Semih Utku(%33),Salih(%33),Utku Enes(%33))
6. Creating a graphic showing the distribution of movies by years and the number of movies(Semih Utku(%33), Utku Enes(%33), Salih Can(%33))
7. Graphic display of the top 20 rated movies(Semih Utku(%33), Utku Enes(%33), Salih Can(%33))
8. Word Cloud Based on Movie Names(Semih Utku(%33), Utku Enes(%33),Murat(%33))
9. Performing operations about seed, train set, test set(Semih Utku(%25), Utku Enes(%25), Salih(%25),Murat(%25))
10. Display the number of rows and the first 200 rows of the training dataset(Semih Utku(%50), Utku Enes(%50))
11. Display the number of rows and the first 200 rows of the test dataset(Semih Utku(%50), Utku Enes(%50))
12. Preparation of a function that creates a recommendation model using the ALS (Alternating Least Squares) algorithm and evaluates the performance of the model(Salih(%33), Semih Utku(%33), Utku Enes(%33))
13. Creating and printing a PySpark DataFrame from the error results list(Salih(%50),Utku Enes(%50))
14. Plotting the MSE (Mean Square Error) and RMSE (Root Mean Square Error) values as a graph(Salih(%50), Utku Enes(%50))
15. Show the model with the lowest error values and the results of this model based on the RMSE and MSE values(Semih Utku(%50), Utku Enes(%50))
16. Cosine Similarity and select the top 10 movies by similarity sum (Murat(%100))
17. Recommendations of users who highly rated other similar movies in the top ten (Murat(%100))
18. Finding 10 users who love product X (Murat(%100))
19. The troubles experienced(Semih Utku(%25), Salih(%25), Utku Enes(%25),Murat(%25))
20. Preparing Report(Semih Utku(%25), Salih(%25), Utku Enes(%25), Murat(%25))

0. Introduction of data

The columns and data types in your table are as follows:

Movie_ID: Integer (integer)

Name: Text (string)

Year: Integer (integer)

User_ID: Integer (integer)

Rating: Integer (integer)

Based on the information above, the metadata information is:

Movie_ID: An integer type column and contains ID numbers that uniquely identify movies.

Name: It is a text (string) type column and contains movie names.

Year: It is an integer type column and contains the movie release years.

User_ID: It is an integer type column and contains ID numbers that uniquely identify users.

Rating: It is an integer type column and contains the ratings that users give to movies.

Movie_ID	Name	Year	User_ID	Rating		
1756	Murder on the Orient Express	1974	6	3		
175	Reservoir Dogs	1992	6	5		
241	North by Northwest	1959	6	3		
1765	First Kid	1996	6	3		
886	Ray	2004	6	5		
1798	Lethal Weapon	1987	6	3		
1799	Die Hard 2: Die Harder	1990	6	3		
468	The Matrix: Revolutions	2003	6	3		
985	The Mummy	1999	6	4		
1803	Alien: Resurrection: Collector's Edition	1997	6	3		
3905	The Others	2001	6	3		
1832	Rosemary's Baby	1968	6	3		
1096	Harlem Nights	1989	6	3		
3315	The Maltese Falcon	1941	6	3		
3893	The People vs. Larry Flynt	1996	6	4		
1843	Beverly Hills Cop III	1994	6	2		
1865	Eternal Sunshine of the Spotless Mind	2004	6	3		
3864	Batman Begins	2005	6	3		
494	Monty Python's The Meaning of Life: Special Edition	1983	6	3		

csv file size: 4 MB, number of lines in the file: 100.000

1. Computer info (with code) IP address, name and configuration of our PC etc.

```
# configurations of PC

import socket
import platform
import re
import uuid
import psutil

# address informations
print('Address:')
print('Ip: ', socket.gethostbyname(socket.gethostname()))
print('Mac: ', ':'.join(re.findall('..', '%012x' % uuid.getnode()))))

print()

# platform informations
print('Platform:')
print('Computer (Host) Name: ', platform.node())
print('OS: ' + platform.system())
print('Release: ' + platform.release())
print('Version: ' + platform.version())
print('Architecture: ' + platform.machine())
print('Processor: ' + platform.processor())

print()

# CPU informations
print('CPU:')
print('Physical Cores', psutil.cpu_count(logical = False)) # except logical cores
print('Total Cores:', psutil.cpu_count(logical = True))
print('Frequency:', psutil.cpu_freq().current / 1000, 'GHz') # megahertz to gigahertz

print()

# memory informations
memory = psutil.virtual_memory()
print('Memory:')
print('Total:', memory.total / 1024**3, 'GB') # bytes to gigabytes
print('Available:', memory.available / 1024**3, 'GB')
print('Used:', memory.used / 1024**3, 'GB')

print()

# disk informations
disk = psutil.disk_usage('/') # root directory (C:\)
print('Disk:')
print('Total:', disk.total / 1024**3, 'GB')
print('Used:', disk.used / 1024**3, 'GB')
print('Free:', disk.free / 1024**3, 'GB')
```

Our goal here is to ensure that the user has information about the hardware and operating system of this computer. (configuration information)

1. The socket module receives information about the network connection, such as IP address and MAC address. The IP address specifies the unique identity of the computer on the network, while the MAC address is a unique identification number assigned to the network interface.

The 2nd platform module is used to obtain information about the operating system. It provides information such as computer name, operating system name, version, architecture, and processor information.

3. The psutil module retrieves information about system resources such as CPU, memory, and disk. CPU information includes physical and total number of cores and processor frequency. Memory information includes information such as the total amount of memory, the amount of available memory, and the amount of memory used. Disk information gives information such as total disk space, used space and free space.

So why do we need this information?

Obtaining this information helps a user monitor their computer's performance, monitor resource usage, and diagnose system problems. This information may also be needed for technical tasks such as software development or network administration. For example, this information can be consulted to determine the system requirements of an application or to monitor devices on the network.

```
Address:
Ip: 192.168.2.100
Mac: 2c:f0:5d:79:72:4e

Platform:
Computer (Host) Name: gamma32
OS: Windows
Release: 10
Version: 10.0.19045
Architecture: AMD64
Processor: AMD64 Family 23 Model 113 Stepping 0, AuthenticAMD

CPU:
Physical Cores 6
Total Cores: 12
Frequency: 3.6 GHz

Memory:
Total: 15.92746353149414 GB
Available: 9.485679626464844 GB
Used: 6.441783905029297 GB

Disk:
Total: 725.8313980102539 GB
Used: 562.6879844665527 GB
Free: 163.14341354370117 GB
```

2. Starting the pyspark session, making the data workable

Relevant libraries are imported: `pyspark.sql` and `pyspark.sql.types`.

`SparkSession` is created: Configured to use a single local node and a thread.

Instant evaluation of Spark SQL statements is enabled.

Specifies the path to the dataset.

The dataset is read and the `DataFrame` is created: The dataset in CSV format is loaded into the `DataFrame` named `users_who_rate`.

Column names are considered the first row.

The data types of the columns are automatically extracted.

Columns are created by defining a specific structure.

The read data set is saved as `DataFrame`.

This code starts Apache Spark by creating a `SparkSession`, reads a dataset and represents it as a Spark `DataFrame`. This `DataFrame` can then be used for data analysis, querying and manipulation.

```

from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType, LongType

from pyspark.sql.functions import split, explode, sum, lit, col, when, asc, desc, avg, mean, count, isnan

# start spark session at single local node with one thread
spark = SparkSession.builder.master('local[1]').appName('191805060_201805051_191805061_201805044').config('spark.driver.extraJavaOptions', '-Xmx1g').getOrCreate()

# enable eager evaluation of Spark SQL expressions in the REPL (Read-Eval-Print Loop)
spark.conf.set('spark.sql.repl.eagerEval.enabled', True)

# path of datasets
path = 'C:/Users/ueb_/Big Data Final(desktop)/netflix.csv'

# read datasets first rows as column names (columns may nullable) and save to DataFrame(df) objects - CSV (Comma Separated Value)
users_who_rate = spark.read \
    .option('header', True) \
    .option('inferSchema', True) \
    .schema(StructType([
        StructField('Movie_ID', LongType(), True),
        StructField('Name', StringType(), True),
        StructField('Year', LongType(), True),
        StructField('User_ID', LongType(), True),
        StructField('Rating', DoubleType(), True),
    ])) \
    .csv(path)

```

3. Recognizing the fact that data preprocessing must occur and taking steps(Salih Can)

```

# is there a NaN or null value in user rating
users_who_rate.select([count(when(isnan('Rating') | col('Rating').isNull(), 'Rating'))])

```

```
count(CASE WHEN (isnan(Rating) OR (Rating IS NULL)) THEN Rating END)
```

0

```

# is there a NaN or null value in movie id
users_who_rate.select([count(when(isnan('Movie_ID') | col('Movie_ID').isNull(), 'Movie_ID'))])

```

```
count(CASE WHEN (isnan(Movie_ID) OR (Movie_ID IS NULL)) THEN Movie_ID END)
```

0

```

# is there a NaN or null value in user id
users_who_rate.select([count(when(isnan('User_ID') | col('User_ID').isNull(), 'User_ID'))])

```

```
count(CASE WHEN (isnan(User_ID) OR (User_ID IS NULL)) THEN User_ID END)
```

0

The `users_who_rate.select()` function allows us to operate on a DataFrame named `users_who_rate`. This DataFrame represents a dataset containing user reviews.

The `count()` function returns the number of rows that satisfy a specified condition.

The `when()` function checks for a given condition and returns a result. In this case, `isnan('Rating') | col('Rating').isNull()` statement checks if the 'Rating' column has a NaN or null value.

As a result, `count(when(isnan('Rating') | col('Rating').isNull(), 'Rating'))` returns the number of rows that have a NaN or null value in the 'Rating' column.

In summary, this snippet checks for such values by returning the number of NaN or null values in the 'Rating' column. (Since the situation is the same in the other two cells, we do not feel the need to explain the other two.)

4. Some information about the dataset we have(Semih Utku,Murat, Salih Can)

```
# columns informations
users_who_rate.printSchema()

print()

users_who_rate.show(200, truncate = True)

# Count the number of unique movie IDs
unique_movie_count = users_who_rate.select("Movie_ID").distinct().count()

# users rates total row count
print('User rates total row count: ', users_who_rate.count())

print()

# Print the result
print("Number of Unique Movie ID: ", unique_movie_count, "\n")

print()

# users rates informations
users_who_rate.select('Rating').describe()
```

First, the schema of the users_who_rate dataset is printed with the printSchema() function. This shows which columns are in the dataset and their data types.

Then a blank line is printed (using the print() function). This is an edit to improve the readability of the output.

The first 200 rows of the users_who_rate dataset are printed with the show() function. This is a process to observe the structure and content of the dataset. The truncate=True parameter improves readability by trimming long lines.

The total number of rows in the dataset is calculated using the users_who_rate.count() function and printed on the screen. This represents the total number of points awarded by users.

Another blank line is printed.

The number of unique movie IDs in the users_who_rate dataset is calculated using the select("Movie_ID").distinct().count() function. This represents the number of different movies included in the dataset. The result is thrown into the "unique_movie_count" variable.

The number of unique movie IDs (unique_movie_count) and a blank line are printed to the screen.

Finally, the statistical summary of the "Rating" column in the users_who_rate dataset is calculated and printed using the select('Rating').describe() function. This shows the statistical distribution of scores given by users.

```

root
|-- Movie_ID: long (nullable = true)
|-- Name: string (nullable = true)
|-- Year: long (nullable = true)
|-- User_ID: long (nullable = true)
|-- Rating: double (nullable = true)

```

Movie_ID	Name	Year	User_ID	Rating
1756	Murder on the Ori...	1974	6	3.0
175	Reservoir Dogs	1992	6	5.0
241	North by Northwest	1959	6	3.0
1765	First Kid	1996	6	3.0
886	Ray	2004	6	5.0
1798	Lethal Weapon	1987	6	3.0
1799	Die Hard 2: Die H...	1990	6	3.0
468	The Matrix: Revol...	2003	6	3.0
4384	Dawn of the Dead	2004	7	1.0
1289	Look Who's Talkin...	1990	7	1.0
4369	Porky's	1981	7	3.0
2783	Hocus Pocus	1993	7	3.0
1307	S.W.A.T.	2003	7	5.0
2782	Braveheart	1995	7	5.0
1314	Presumed Innocent	1990	7	4.0
1428	The Recruit	2003	7	4.0
4306	The Sixth Sense	1999	7	5.0
2755	The Longest Yard	1974	7	3.0
1359	Funny Lady	1975	7	5.0
1406	Hook	1991	7	3.0
4266	The Passion of th...	2004	7	5.0

only showing top 200 rows

User rates total row count: 100000

Number of Unique Movie ID: 1350

summary	Rating
count	100000
mean	3.61908
stddev	1.0588725018219113
min	1.0
max	5.0

5. The process of showing how many users are for each rating count

```

# rating distribution

# group by user rating and count the number of user ratings
rating_distribution = users_who_rate.groupby('Rating').count().orderBy('Rating')

# convert pyspark data frame to pandas data frame for plotting with seaborn
rating_distribution_pandas = rating_distribution.toPandas()

plt.figure(figsize = (12, 8))
sns.barplot(x = 'Rating', y = 'count', data = rating_distribution_pandas, palette = 'Set1')
plt.title('Rating Distribution', size = 20)
plt.xlabel('Rating Numbers')
plt.ylabel('Count of Ratings')
plt.show()

```

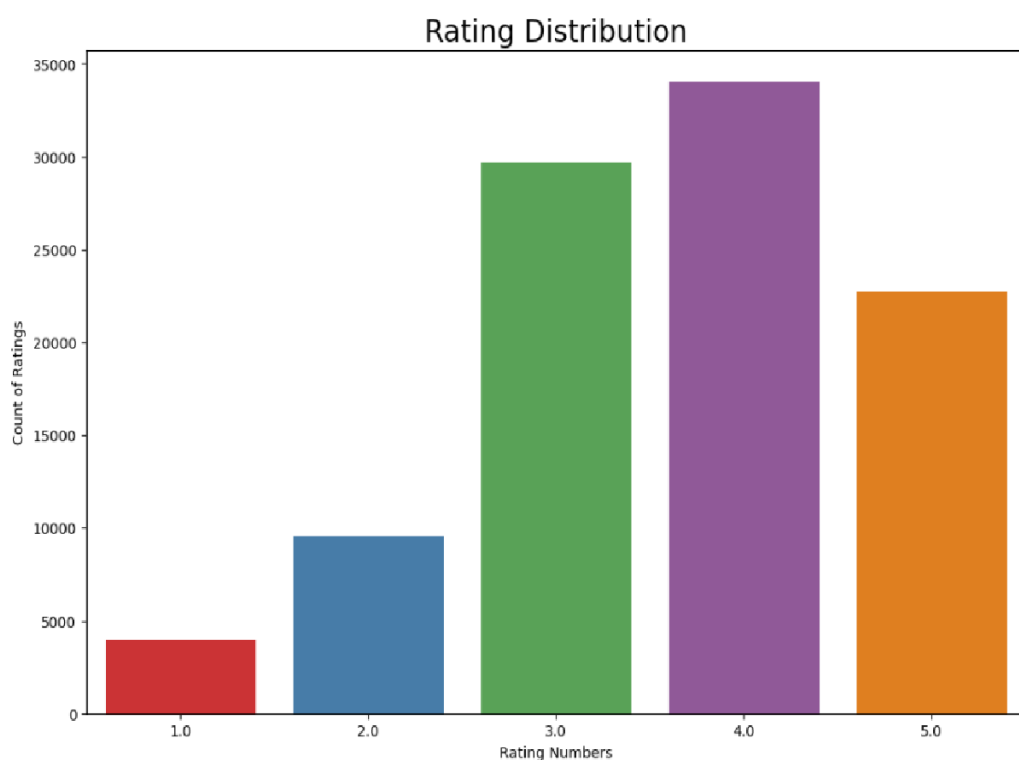
This code is used to visualize the distribution of ratings given by users. First, a PySpark dataframe called `users_who_rate` contains user reviews. This data frame should contain information about users' evaluations.

Next, we group by user ratings using the groupby function, and calculate and rank the number of ratings for each rating.

In the next step, we convert the PySpark dataframe to a pandas dataframe that can be used in the seaborn library (using the toPandas() function).

Next, we create a bar chart using the seaborn and matplotlib libraries. The x-axis represents the user review scores, while the y-axis represents the number of each review score.

Finally, we show the bar chart (plt.show()) by adjusting the chart title, the missing and the size of the image.



6. Creating a graphic showing the distribution of movies by years and the number of movies

`users_who_rate.filter(users_who_rate['Year'] >= 1968)`: In the first step, we filter by the 'Year' column in a dataframe named 'users_who_rate'. The filtering condition is that it only includes films from 1968 and later.

As a result of this filtering, we get a data frame containing the movies voted by users from the years after 1968. `groupby('Year').count()`: In the second step, we group by the 'Year' column. This grouping process is used to count how many movies are available for each year. As a result, we get a data frame containing the number of movies for each year. `orderBy('Year')`: In the third step, we sort by

the 'Year' column. This allows us to arrange the years in ascending order.
number_of_movies_by_year.toPandas():

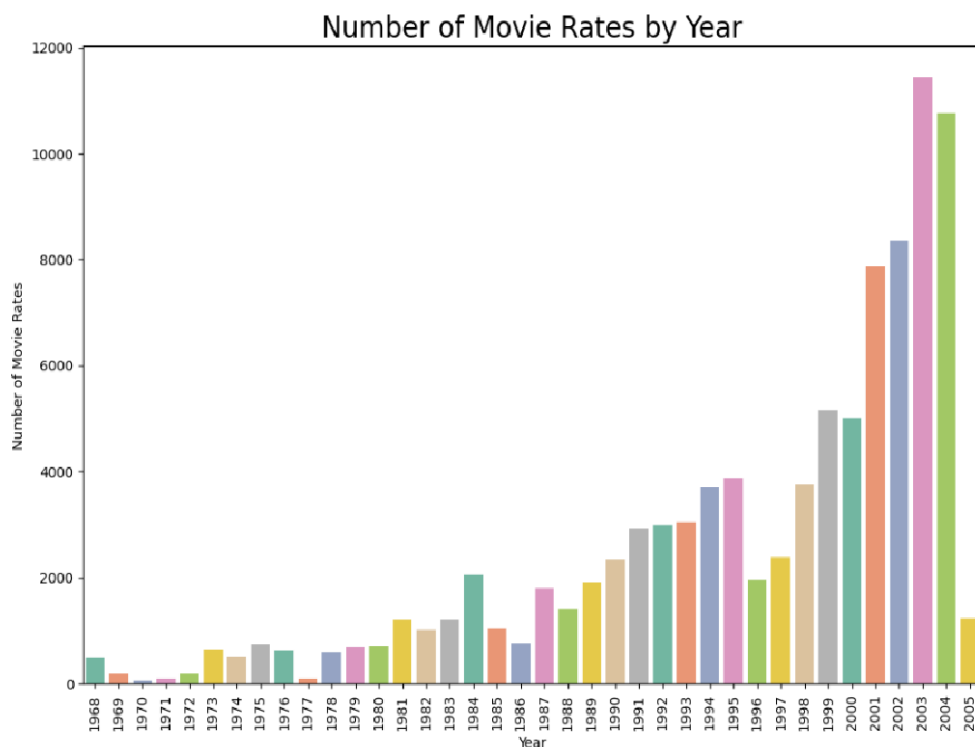
In the fourth step, we convert the PySpark dataframe to Pandas dataframe. This is necessary as we will use the data later to visualize it with the Seaborn library. sns.barplot(x = 'Year', y = 'count', data = number_of_movies_by_year_pandas, palette = 'Set2'): In the fifth step, we create a bar chart using the Seaborn library. We use the 'year' column as the x-axis and the 'count' column as the y-axis. We get the data from the 'number_of_movies_by_year_pandas' dataframe. We define the colors using the 'Set2' palette. Other plt functions: The remaining plt functions are used to customize the chart with title, labels and other display settings. plt.show(): Displays the chart.

```
# number of movies by year

# group by year and count the number of movies after 1968
number_of_movies_by_year = users_who_rate.filter(users_who_rate['Year'] >= 1968).groupby('Year').count().orderBy('Year')

# convert pyspark data frame to pandas data frame for plotting with seaborn
number_of_movies_by_year_pandas = number_of_movies_by_year.toPandas()

plt.figure(figsize = (12, 8))
sns.barplot(x = 'Year', y = 'count', data = number_of_movies_by_year_pandas, palette = 'Set2')
plt.title('Number of Movie Rates by Year', size = 20)
plt.xlabel('Year')
plt.xticks(rotation = 90)
plt.ylabel('Number of Movie Rates')
plt.show()
```



7. Graphic display of the top 20 rated movies

```
# Group by movie name and sum user ratings
movie_rating_distribution = users_who_rate.groupby('Name').sum('Rating')

# Convert to pandas dataframe and sort by the sum of user ratings
movie_rating_distribution_pandas = movie_rating_distribution.toPandas().sort_values(by = 'sum(Rating)', ascending = False)[:20]

plt.figure(figsize = (12, 8))
sns.barplot(x = 'sum(Rating)', y = 'Name', data = movie_rating_distribution_pandas, palette = 'Set3')
plt.title('Top 20 Rated Movies', size = 20)
plt.xlabel('Total Rating Scores')
plt.ylabel('Movie Name')
plt.show()
```

This piece of code aims to group movie ratings in a data frame, collect user ratings by movie name, and then visualize the top 20 movies with the highest overall ratings. It performs operations with the following steps:

`users_who_rate.groupby('Name').sum('Rating')`: Groups the 'users_who_rate' dataframe by the 'Name' column and aggregates the 'Rating' column. This is used to get the total user rating for each movie. The result is returned as a PySpark DataFrame and assigned to a variable named 'movie_rating_distribution'.

`movie_rating_distribution.toPandas()`: Convert 'movie_rating_distribution' DataFrame to Pandas DataFrame. This is used to more easily manipulate and visualize data.

`sort_values(by='sum(Rating)', ascending=False)`: Sorting is done by column 'sum(Rating)' in Pandas DataFrame. This is used to show the movies with the highest overall ratings at the top.

`[:20]`: Selects the first 20 rows from the sorted DataFrame. This is used to get the movies with the top 20 overall ratings.

`plt.figure(figsize=(12, 8))`: A 12x8 inch plot area is created. This creates a visual field for the bar chart.

`sns.barplot(x='sum(Rating)', y='Name', data=movie_rating_distribution_pandas, palette='Set3')`: Barplot is created using the Seaborn library's barplot function. The 'sum(Rating)' column is used as the horizontal axis (x-axis) and the 'Name' column is used as the vertical axis (y-axis). The data is taken from the Pandas DataFrame 'movie_rating_distribution_pandas'. The 'palette' parameter is used to specify the color palette of the bars.

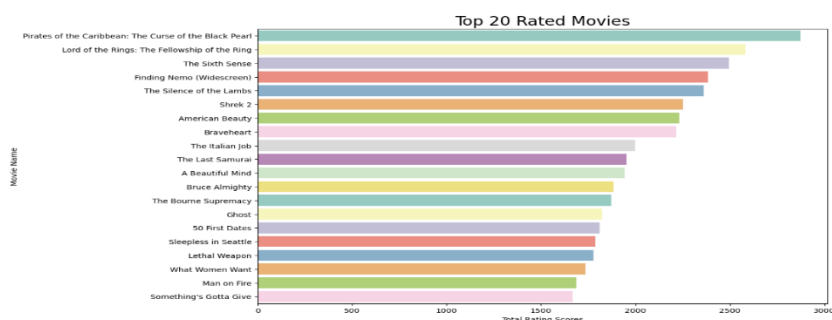
`plt.title('Top 20 Rated Movies', size=20)`: Graphic title is determined.

`plt.xlabel('Total Rating Scores')`: X-axis label is determined.

`plt.ylabel('Movie Name')`: Y-axis label is set.

`plt.show()`: The created bar chart is displayed.

This piece of code collects movie ratings and aims to present the most popular movies to users by visualizing the movies with the highest overall ratings.



8. Word Cloud Based on Movie Names

```
from wordcloud import WordCloud

plt.subplots(figsize=(30, 15))
wordcloud = WordCloud(background_color='aliceblue', width=1920, height=1080).generate(" ".join(movie_rating_distribution_pandas['Name']))
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

First, we import the WordCloud class using the `from wordcloud import WordCloud` statement to create an object named `wordcloud`.

The statement `plt.subplots(figsize=(30, 15))` imports the `matplotlib.pyplot` module as `plt` and creates a graphic figure. The `figsize=(30, 15)` parameter sets the figure's width to 30 units and its height to 15 units.

When creating the WordCloud object, we give the value `'aliceblue'` to the `background_color` parameter. This sets the background color of the word cloud.

The width and height parameters determine the width and height of the word cloud, in pixels. This example uses 1920 pixels wide and 1080 pixels high.

The `generate` method gets the text to be displayed in the word cloud. In this example, we create a text by combining `movie_rating_distribution_pandas['Name']` with all values in the "Name" column in a dataframe named `"movie_rating_distribution_pandas"`.

The `plt.imshow(wordcloud)` statement creates the word cloud using the `wordcloud` object and adds it to the graphic figure.

The `plt.axis('off')` statement turns off the axes (x and y axes) so that the axes do not appear in the word cloud output.

Finally, the word cloud created by the `plt.show()` method is displayed.

This code creates a word cloud using words in a text and displays the generated word cloud using the `matplotlib` library. The display frequency of words varies depending on how often they are used in the text and is represented by the display size of the word.

9. Performing operations about seed, train set, test set

Seed is a random starting point in machine learning algorithms. The same random numbers can be generated using the same seed value. Seed's intended use is to eliminate randomness inconsistencies and ensure repeatability of results. Thus, the behavior of the algorithm becomes more consistent. The underlying logic of Seed is to ensure the repeatability of the algorithm and to evaluate the performance of the algorithm by experimenting using different seed values.

Train set is the data set in which the model is trained and learns the patterns.

The **test set** is the data set in which the performance of the trained model is evaluated and its generalization ability is measured.

```
# recommendation engine

# use 70% of dataset for training, rest of them for testing
# randomly selected rows from users_who_rate_for_prediction with reference of seed
# data will split in the same way every time with same seed (use seed for consistent results)
train_data, test_data = users_who_rate.randomSplit([0.7, 0.3], seed = 5061)
```

The `randomSplit([0.7, 0.3], seed = 5061)` command randomly splits the dataset into 70% training set and 30% test set. That is, 70% of the dataset is used for training, while the remaining 30% is used for testing.

The seed value (`seed = 5061`) specifies the starting point for randomness. This value ensures that the dataset is split in the same way every time the same seed value is used. This makes the results reproducible and ensures consistency between experiments.

As a result, two datasets called `train_data` and `test_data` are created. `train_data` represents the dataset used for 70% training, while `test_data` represents the dataset used for 30% testing. These splits are performed in the same way, using the same seed value each time.

10. Display the number of rows and the first 200 rows of the training dataset

```
# train data total row count
print('Train data total row count: ', train_data.count())

print()

train_data = train_data.orderBy('User_ID', ascending = True)
train_data.show(200, truncate = True)
```

The `train_data.count()` command returns the total number of rows in the `train_data` dataset. This number represents the total number of samples in the data set. This value indicates the size of the training dataset.

Next, the `train_data` dataset is sorted ascending by column 'User_ID'. The `orderBy('User_ID', ascending=True)` statement causes the dataset to be sorted by the 'User_ID' column. The 'ascending=True' parameter specifies that sorting is done in ascending order.

Finally, the `train_data.show(200, truncate=True)` command prints the first 200 lines of the `train_data` dataset, clipped with the 'truncate=True' parameter. This is used to view samples of the dataset. The 'truncate=True' parameter allows long lines in the dataset to be truncated.

1238	The Pope of Green...	1984	6	2.0
173	The Devil's Brigade	1968	6	4.0
40	Something's Gotta...	2003	6	3.0
1971	Under Siege 2: Da...	1995	6	3.0
2000	Four Weddings and...	1994	6	3.0
2001	Under Siege	1997	6	4.0
2095	Liar Liar	1997	6	4.0
2112	Identity	2003	6	3.0
1267	Dr. Dolittle 2	2001	6	3.0
241	North by Northwest	1959	6	3.0
329	Dogma	1999	6	4.0
564	Predator 2	1990	6	4.0
3254	Daredevil	2003	7	5.0
1066	Superman: The Movie	1978	7	5.0
1144	Fried Green Tomatoes	1991	7	5.0
1180	A Beautiful Mind	2001	7	5.0
299	Bridget Jones's D...	2001	7	4.0
2743	The Pianist	2002	7	5.0
28	Lilo and Stitch	2002	7	4.0
1754	Sixteen Candles	1984	7	3.0
1810	U.S. Marshals	1998	7	2.0
1918	The Alamo	2004	7	5.0
348	The Last Temptati...	1988	7	4.0
708	The Perfect Storm	2000	7	3.0
2290	Aladdin: Platinum...	1992	7	4.0
83	Silkwood	1983	7	5.0
2862	The Silence of th...	1991	7	5.0
3071	The Way We Were	1973	7	5.0

+-----+-----+-----+-----+
only showing top 200 rows

11. Display the number of rows and the first 200 rows of the test dataset

```
# test data total row count
print('Test data total row count: ', test_data.count())

print()

test_data = test_data.orderBy('User_ID', ascending = True)
test_data.show(200, truncate = True)
```

The `test_data.count()` command returns the total number of rows in the `test_data` dataset. This number represents the total number of samples in the data set. This value indicates the size of the test data set.

Next, the `test_data` dataset is sorted ascending by column 'User_ID'. The `orderBy('User_ID', ascending=True)` statement causes the dataset to be sorted by the 'User_ID' column. The 'ascending=True' parameter specifies that sorting is done in ascending order.

Finally, the `test_data.show(200, truncate=True)` command prints the first 200 lines of the `test_data` dataset, clipped with the `'truncate=True'` parameter. This is used to view samples of the dataset. The `'truncate=True'` parameter allows long lines in the dataset to be truncated.

Test data total row count: 30155

Movie_ID	Name	Year	User_ID	Rating
723	Curly Sue	1991	6	3.0
3689	The Amityville Ho...	2005	6	3.0
2340	Joe Versus the Vo...	1990	6	3.0
3860	Bruce Almighty	2003	6	3.0
3894	Surviving Christmas	2004	6	2.0
1467	Three Musketeers	1993	6	5.0
2342	Super Size Me	2004	6	3.0
4149	Scooby-Doo 2: Mon...	2004	6	4.0
4306	The Sixth Sense	1999	6	4.0
2372	The Bourne Supremacy	2004	6	5.0
4393	The Mask: Special...	1994	6	3.0
4432	The Italian Job	2003	6	3.0
494	Monty Python's Th...	1983	6	3.0
1180	A Beautiful Mind	2001	97	5.0
896	Dangerous Minds	1995	97	4.0
483	Rush Hour 2	2001	97	1.0
1144	Fried Green Tomatoes	1991	97	5.0
2551	Trigun	1999	97	1.0
4216	Jurassic Park III	2001	97	3.0
4208	Code of Silence	1985	97	1.0
2122	Being John Malkovich	1999	97	2.0
3466	Spy Kids	2001	97	3.0
2452	Lord of the Rings...	2001	97	5.0
83	Silkwood	1983	97	4.0
4356	Road to Perdition	2002	97	4.0
148	Sweet November	2001	134	5.0

only showing top 200 rows

12. Preparation of a function that creates a recommendation model using the ALS (Alternating Least Squares) algorithm and evaluates the performance of the model

```
def prediction(rank, maxIter, regParam):  
  
    # ALS  
    als = ALS(userCol = 'User_ID', itemCol = 'Movie_ID', ratingCol = 'Rating', seed = 5061, implicitPrefs = False, nonnegative =  
    model = als.fit(train_data)  
    predictions = model.transform(test_data)  
    predictions = predictions.withColumnRenamed('prediction', 'Prediction')  
  
    # evaluate model  
  
    # rmse (root mean squared error)  
    rmse_evaluator = RegressionEvaluator(metricName = 'rmse', labelCol = 'Rating', predictionCol = 'Prediction')  
    rmse = rmse_evaluator.evaluate(predictions)  
  
    # mse (mean squared error)  
    mse_evaluator = RegressionEvaluator(metricName = 'mse', labelCol = 'Rating', predictionCol = 'Prediction')  
    mse = mse_evaluator.evaluate(predictions)  
  
    return {'model': model, 'predictions': predictions, 'error_results': {'rank': rank, 'iteration': maxIter, 'lambda': regParam,
```

While writing this function, we aimed to create a recommendation model using the ALS algorithm and evaluate the performance of the model.

We used this function, which we wrote for ALS, for 18 models, but of course, we will only show one as an example in our report.

Estimates are made: test_data represents the test dataset. The model.transform() method uses the model to make predictions on the test dataset. In these rows, a DataFrame containing the results of the predictions is created and the prediction column name is renamed to 'Prediction'.

defines a function

```
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.recommendation import ALS

def prediction(rank, maxIter, regParam):
    # ALS
    als = ALS(userCol = 'User_ID', itemCol = 'Movie_ID', ratingCol = 'Rating', seed = 5061, implicitPrefs = False, nonnegative = True)
    model = als.fit(train_data)
    predictions = model.transform(test_data)
    predictions = predictions.withColumnRenamed('prediction', 'Prediction')

    # evaluate model
    # rmse (root mean squared error)
    rmse_evaluator = RegressionEvaluator(metricName = 'rmse', labelCol = 'Rating', predictionCol = 'Prediction')
    rmse = rmse_evaluator.evaluate(predictions)

    # mse (mean squared error)
    mse_evaluator = RegressionEvaluator(metricName = 'mse', labelCol = 'Rating', predictionCol = 'Prediction')
    mse = mse_evaluator.evaluate(predictions)

    return {'model': model, 'predictions': predictions, 'error_results': {'rank': rank, 'iteration': maxIter, 'lambda': regParam, 'RMSE': rmse, 'MSE': mse}}
```

RegressionEvaluator, modelin performansını değerlendirmek için kullanılırken, ALS modeli, işbirliği filtrelemesi (collaborative filtering) algoritmasını uygulamak için kullanılır.

Als creates the model
The ALS model is defined using the ALS class. In this line, the parameters and settings of the ALS model are specified. For example, we set the user column as 'User_ID', the item column as 'Movie_ID' and the rating column as 'Rating'.

train_data represents the training dataset. The als.fit() method is used to train the ALS model. The model learns the user and item factors on the training data.

The model is evaluated: The RegressionEvaluator class is used to evaluate the performance of the model. In these lines, the error values of the predictions are calculated using the RMSE (Root Mean Squared Error) and MSE (Mean Squared Error) measurements.

Note: In order to better understand this function, we decided to explain the part it returns in more detail.

'model': model: This pair represents the trained ALS model. The model variable holds the ALS model object that has been trained with the als.fit() method. In this way, you can use the trained model outside the function.

'predictions': predictions: This pair represents a DataFrame that contains the results of the predictions made on the test dataset. The predictions variable contains the results of the predictions obtained by the model.transform() method. This DataFrame may contain information such as movies recommended to users or rating estimates.

'error_results': {'rank': rank, 'iteration': maxIter, 'lambda': regParam, 'RMSE': rmse, 'MSE': mse}: This pair represents an internal dictionary containing the model's evaluation results and the parameters used it does. The internal dictionary imports the configuration parameters of the model such as rank, maxIter and regParam and the RMSE and MSE error values calculated with these parameters.

```
def add_model_results(result_dict, model_name, error_results):
    """
    Verilen model adı ve hata sonuçlarına göre "result_dict" sözlüğüne sonuçları ekleyen fonksiyon.
    :param result_dict: Sonuçların ekleneceği sözlük.
    :param model_name: Modelin adı.
    :param error_results: Modelin hata sonuçları sözlüğü.
    :return: None
    """
    result_dict[model_name] = {'RMSE': error_results['RMSE'], 'MSE': error_results['MSE']}

result_dict = {}

model_name = 'model_1_1'
error_results = model_1_1['error_results']
add_model_results(result_dict, model_name, error_results)
```

This code defines a function that adds the results to a dictionary, based on a given dictionary of model names and error results.

The function is called "add_model_results" and has three parameters:

"result_dict": The dictionary into which the results will be inserted.

"model_name": The name of the model.

"error_results": The model's dictionary of error results.

(we will talk about this function again later)

To better understand ALS, let's look at its input and output.

```
{'rank': 10, 'iteration': 10, 'lambda': 0.01, 'RMSE': 0.9978891324266991, 'MSE': 0.9957827206153103}
```

Movie_ID	Name	Year	User_ID	Rating	Prediction
290	Harold and Kumar ...	2004	10817	3.0	3.1750705
345	Star Trek: Voyage...	1998	10817	4.0	3.9380772
788	Clerks	1994	10817	4.0	3.9357905
940	Hercules	1997	6336	2.0	2.572569
1220	Man on Fire	2004	10817	4.0	3.9951267
1336	Wolf	1994	10817	3.0	3.1288185
1518	Casper	1995	10817	3.0	3.0140421
1703	Ever After: A Cin...	1998	10817	3.0	3.6151667
1905	Pirates of the Ca...	2003	10817	3.0	4.1515675
1962	50 First Dates	2004	6336	3.0	3.9998927
1975	Hollow Man	2000	6336	1.0	3.4914238
2095	Liar Liar	1997	10817	2.0	3.5938427
457	Kill Bill: Vol. 2	2004	7152	4.0	4.309302
457	Kill Bill: Vol. 2	2004	9597	3.0	2.3025212
473	Princess Mononoke	1997	7576	5.0	4.237632
548	Hudson Hawk	1991	7576	5.0	2.7079995
571	American Beauty	1999	5682	5.0	3.8957195
571	American Beauty	1999	6500	4.0	3.89042
571	American Beauty	1999	7152	4.0	4.039211
571	American Beauty	1999	7576	3.0	4.347825

only showing top 200 rows

Since 17 different models like this model were carried out, we do not show the rest in our report by making 1 model example (with "iteration" , "rank" and "lambda" expressions changed).

13. Creating and printing a PySpark DataFrame from the error results list

First, an empty list named `error_results` is defined.

Then, with statements like `error_results.append(model_1_1['error_results'])`, error results of different models (values with key 'error_results') are added to the list of `error_results` respectively.

Here, `model_1_1`, `model_1_2`, `model_1_3`, etc. We assume that there are variables with different model outcomes.

In the next step, a PySpark DataFrame is created from the `error_results` list using the `spark.createDataFrame(error_results)` statement. This will convert the data in the `error_results` list to DataFrame format.

Finally, the `error_results_data_frame` statement can be used to print the DataFrame assigned to the `error_results_data_frame` variable.

```
error_results.append(model_2_1['error_results'])
error_results.append(model_2_2['error_results'])
error_results.append(model_2_3['error_results'])
error_results.append(model_3_1['error_results'])
error_results.append(model_3_2['error_results'])
error_results.append(model_3_3['error_results'])

error_results.append(model_1_1_1['error_results'])
error_results.append(model_1_2_2['error_results'])
error_results.append(model_1_3_3['error_results'])
error_results.append(model_2_1_1['error_results'])
error_results.append(model_2_2_2['error_results'])
error_results.append(model_2_3_3['error_results'])
error_results.append(model_3_1_1['error_results'])
error_results.append(model_3_2_2['error_results'])
error_results.append(model_3_3_3['error_results'])

error_results_data_frame = spark.createDataFrame(error_results)
error_results_data_frame.show(200, truncate = True)
```

	MSE	RMSE	iteration	lambda	rank
0.9957827206153103	0.9978891324266991	10	0.01	10	
1.0024478080645116	1.0012231559769837	50	0.01	10	
1.0202302031318535	1.010064454939314	200	0.01	10	
1.1593764905917983	1.0767434655440442	10	0.01	50	
1.2261283199911963	1.107306786753877	50	0.01	50	
1.234557814406653	1.1111065720292779	200	0.01	50	
1.096178507691285	1.0469854381467227	10	0.01	200	
0.9315734038451456	0.9651805032454529	50	0.01	200	
0.861030951656638	0.9279175349440477	200	0.01	200	
0.7932308733729988	0.8906350955206059	10	0.1	10	
0.7837614754435867	0.8853030415872221	50	0.1	10	
0.7839000016656434	0.8853812747430586	200	0.1	10	
0.7838331756626356	0.8853435353932594	10	0.1	50	
0.7820743326599511	0.8843496665120371	50	0.1	50	
0.7822105738185247	0.8844266921676011	200	0.1	50	
0.7828866441071105	0.8848088178285242	10	0.1	200	
0.7801563746442631	0.8832646119053242	50	0.1	200	
0.7804097229234983	0.8834080161077883	200	0.1	200	

14. Plotting the MSE (Mean Square Error) and RMSE (Root Mean Square Error) values as a graph

```
# convert pyspark data frame to pandas data frame
error_results_pandas = error_results_data_frame.toPandas()

# Plot MSE and RMSE
plt.plot(error_results_pandas.index, error_results_pandas['MSE'], linestyle='--', color = 'red', label = 'MSE')
plt.plot(error_results_pandas.index, error_results_pandas['RMSE'], linestyle='--', color = 'orange', label = 'RMSE')

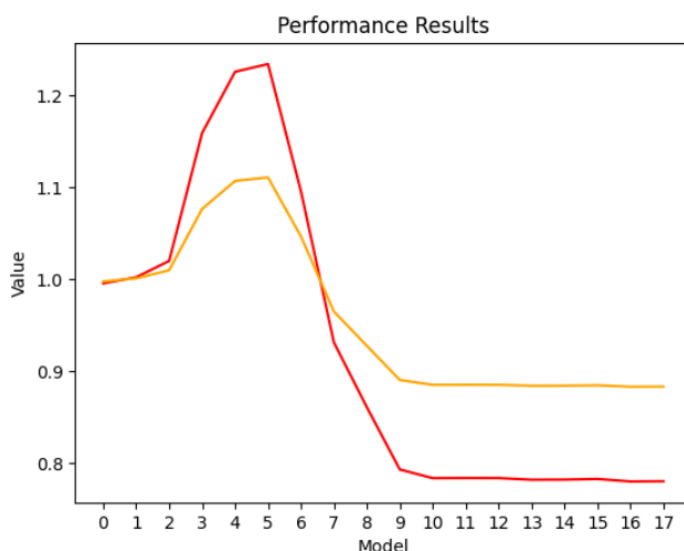
# plt.plot(error_results_pandas.index, error_results_pandas['rank'], linestyle='--', color = 'purple', label = 'Rank')
# plt.plot(error_results_pandas.index, error_results_pandas['iteration'], linestyle='--', color = 'black', label = 'Iteration')
# plt.plot(error_results_pandas.index, error_results_pandas['lambda'], linestyle='--', color = 'brown', label = 'Lambda')

plt.xlabel('Model')
plt.xticks(error_results_pandas.index.astype(int))
plt.ylabel('Value')
plt.title('Performance Results')
plt.figure(figsize = (12, 8))
plt.show()
```

we aimed to convert a PySpark dataframe (DataFrame) to a Pandas dataframe and then display error results such as MSE (Mean Squared Error) and RMSE (Root Mean Squared Error) on a graph.

In the first line, we assume there is a PySpark dataframe named `error_results_data_frame` and we convert this dataframe into a Pandas dataframe using the `toPandas()` method. The converted dataframe is assigned to the variable `error_results_pandas`.

In the next steps, two separate line plots representing the MSE and RMSE values are plotted using the `plt.plot()` function. The values of columns 'MSE' and 'RMSE' are plotted on the x-axis using `error_results_pandas.index` and on the y-axis using the values in columns 'MSE' and 'RMSE', respectively. The `linestyle` and `color` parameters are used to specify the line style and color.



15. Show the model with the lowest error values and the results of this model based on the RMSE and MSE values

This code aims to find the best model using the `model_list` and `result_dict` variables, which have a set of models and the performance results of those models. The best model is determined based on the root mean square error (RMSE) and mean square error (MSE) values.

First, the variables `best_rmse` and `best_mse` are set to infinity (`float('inf')`). This is greater than any actual value and represents the best initial RMSE and MSE values.

The `best_result_index` variable is initially set to `None` and is used to track which model has the best result.

A loop is then created for each model. At each loop step, the results of the model in `result_dict` are retrieved and assigned to the variables `rmse` and `mse`.

Then, if the `rmse` value is less than or equal to `best_rmse` and the `mse` value is less than or equal to `best_mse`, it means this model performs better. In this case, `best_rmse` is updated with the value of `rmse`, `best_mse` is updated with the value of `mse`, and the variable `best_result_index` is updated with the `i` representing the index of this model with the best results.

After the loop is complete, the best model is accessed from the `model_list` using `best_result_index` and assigned as the `best_model`.

Finally, using `result_dict`, the results of the best model are printed to the screen, and the name of the best model (`best_model`) with its description text is printed to the screen.

This code aims to reach the best model by comparing the RMSE and MSE values of each model in the `model_list`. The lower the RMSE and MSE values indicate how much the model reduces prediction errors. Lower values indicate that the model's predictions are closer to the true values and perform better.

```
model_listesi = ['model_1_1', 'model_1_2', 'model_1_3', 'model_2_1', 'model_2_2', 'model_2_3', 'model_3_1', 'model_3_2', 'model_3_3']

best_rmse = float('inf')
best_mse = float('inf')
best_result_index = None

# Her bir model için döngü
for i, model in enumerate(model_listesi):
    result = result_dict[model]
    rmse = result['RMSE']
    mse = result['MSE']

    if rmse <= best_rmse and mse <= best_mse:
        best_rmse = rmse
        best_mse = mse
        best_result_index = i

best_model = model_listesi[best_result_index]

print('Best model based on RMSE and MSE:', result_dict[best_model])
print(f'Best model is: {best_model}', "\n")
print("The best model is selected based on having the lowest RMSE or MSE, as these indicate the model's ability to minimize prediction errors.")

Best model based on RMSE and MSE: {'RMSE': 0.8832646119053242, 'MSE': 0.7801563746442631}
Best model is: model_3_2_2
```

The best model is selected based on having the lowest RMSE or MSE, as these indicate the model's ability to minimize prediction errors. Lower values suggest that the model's predictions are closer to the actual values, resulting in better overall performance.

16. Cosine Similarity and select the top 10 movies by similarity sum

```
from pyspark.sql import Row
from numpy import linalg as LA
from pyspark.sql import functions as F
import numpy as np

# In collaborative filtering and recommendation systems, the itemFactors represents the learned latent factors...
# ...for the items in the model. In collaborative filtering, latent factors are used to represent the preferences or...
# ...properties of users and items (products, movies, etc.). So we used 'itemFactors'. \\\ itemFactors has movie_id-as id and f
best_model_item_factors = my_dict['model'].itemFactors

# Determine movie X
movie_id = 1843
other_movies = []

# itemFactors uygulanmis halin id'si ile movie_id'nin ayni oldugu filmleri selected movie olarak atanmasi
selected_movie = best_model_item_factors.where(best_model_item_factors.id == movie_id).select('features')

# selected_movie_features secilen X filminin ALS'de hesaplanan urun-film faktorleri-RDD:Resilient Distributed Dataset
selected_movie_features = selected_movie.rdd.map(lambda x: x.features).first()

# The toLocalIterator() function returns an iterator that allows you to sequentially iterate over the elements...
# ...of the RDD on the driver program without bringing the entire RDD into memory at once.
for row in best_model_item_factors.rdd.toLocalIterator():
    # Ayni filmin similarity sini hesaplamak icin...
    if row.id != movie_id:
        #v1 diger filmler vektoru
        v1 = np.asarray(row.features)
        #v2 secilen X filminin vektoru
        v2 = np.asarray(selected_movie_features)
        #cosine similarity formulu
        similarity_score = v1.dot(v2) / (LA.norm(v1) * LA.norm(v2))
        #diger filmlerin id'si ile similarity_score'lari, other_movies adli listede tutma
        other_movies.append([row.id, similarity_score])
R = Row('Movie_ID', 'Similarity_Score')

#other_movies liste seklinde, asagidaki kod ile onu dataframe haline getirdik - ikisi de ayni verileri iceriyor
similar_items_dataframe = spark.createDataFrame([R(column[0], float(column[1])) for column in other_movies])

# Aggregate the similarity scores using reduceByKey()
# Aggregation is a process in data analysis where data is grouped together and summarized to provide meaningful insights.
similarity_sum_rdd = similar_items_dataframe.rdd.map(lambda x: (x['Movie_ID'], x['Similarity_Score'])).reduceByKey(lambda a, b:

# Create a new DataFrame from the aggregated RDD
R = Row('Movie_ID', 'Similarity_Sum')
similarity_sum_dataframe = spark.createDataFrame([R(movie_id, similarity_sum) for movie_id, similarity_sum in similarity_sum_rdd
```

The code provided performs item-based collaborative filtering using Spark's DataFrame and RDD APIs. It starts by accessing the item factors from a 'model' object. Then, a specific movie (X) is selected based on its movie ID, and its features are retrieved. Next, the code iterates over all other movies' features, calculating the cosine similarity score between each movie and X. These similarity scores are stored in a list called 'other_movies'.

Afterward, a DataFrame named 'similar_items_dataframe' is created from the 'other_movies' list, containing movie IDs and similarity scores. The similarity scores are then aggregated by summing them for each movie ID using the 'reduceByKey()' operation on the RDD. Finally, a new DataFrame called 'similarity_sum_dataframe' is created from the aggregated scores.

In summary, the code calculates the similarity scores between a selected movie (X) and all other movies based on their latent factors. It aggregates the similarity scores and produces a DataFrame that contains the movie IDs and their respective aggregated similarity sums. This approach can be used in recommendation systems to find movies that are similar to a given movie.

```
# Select the top 10 movies by similarity sum
top_ten_movies = similarity_sum_dataframe.orderBy(similarity_sum_dataframe.Similarity_Sum.desc()).limit(10)
# Show the top 10 movies
top_ten_movies.show()
```

```
+-----+-----+
|Movie_ID| Similarity_Sum|
+-----+-----+
| 1216| 0.8600693299660453|
| 629| 0.8595112100966648|
| 1530| 0.8548991935887772|
| 3830| 0.8546425389028651|
| 406| 0.8537344825526306|
| 3229| 0.8528497769522815|
| 1174| 0.8465830071919102|
| 3826| 0.8449957827719949|
| 454| 0.8437066456831962|
| 3610| 0.840890976742002|
+-----+-----+
```

top_ten_movies =

similarity_sum_dataframe.orderBy(similarity_sum_dataframe.Similarity_Sum.desc()).limit(10): This line sorts the similarity_sum_dataframe DataFrame in descending order based on the values in the "Similarity_Sum" column. orderBy is used to perform the sorting, and desc() specifies that the sorting should be in descending order. The limit(10) function is then used to select only the top 10 rows from the sorted DataFrame. The result is assigned to the top_ten_movies variable.

top_ten_movies.show(): This line displays the contents of the top_ten_movies DataFrame. The show() function is a method in PySpark (assuming you're using PySpark) that prints the DataFrame in a tabular format to the console.

17. Recommendations of users who highly rated other similar movies in the top ten

```
# Returns list
top_ten_movie_ids = top_ten_movies.rdd.map(lambda x: x.Movie_ID).collect()
```

```
# Secilen X filminin ratingini veren kullanicilari filtreleyip bir dataframe'e koyduk
users_who Rated_selected_movie = my_dict['predictions'].filter(my_dict['predictions'].Movie_ID == movie_id)
user_ids_who Rated_selected_movie = users_who Rated_selected_movie.select("User_ID").rdd.flatMap(lambda x: x).collect()
# Secilen X filminin rating vermeyen userlari filtreleyip bir dataframe'e koyduk
users_without Rated_selected_movie = my_dict['predictions'].filter(~my_dict['predictions'].User_ID.isin(user_ids_who Rated_selected_movie))
```

```
recommended_users = []
for new_movie_id in top_ten_movie_ids:
    recommended_selected_movie = my_dict['predictions'].filter(my_dict['predictions'].Movie_ID == new_movie_id)
    sorted_ratings = recommended_selected_movie.sort(F.desc('Rating'))
    users_who_like_selected_movie = sorted_ratings.select('User_ID').limit(10)
    users_who_like_selected_movie_ids = users_who_like_selected_movie.rdd.map(lambda x: x.User_ID).collect()
    recommended_users.append({'Movie_ID': new_movie_id, 'User_ID': users_who_like_selected_movie_ids})

recommended_dataframe = spark.createDataFrame(recommended_users)
recommended_dataframe.show()
```

The code performs several steps to recommend users for movies. First, it collects the top ten movie IDs from a DataFrame into a Python list. Then, it filters the DataFrame to include only users who have rated a specific movie and extracts their user IDs. Next, it filters the DataFrame to exclude users who have already rated the selected movie. Moving on, the code recommends users for each movie in the top ten list by selecting the top 10 user IDs with the highest ratings for each movie. Finally, it creates a DataFrame of the recommended users and displays its contents using the show() function. Overall, the code collects movie IDs, filters users who rated a specific movie, recommends users for each movie, and creates a DataFrame of recommended users.

18. Finding 10 users who love product x

```
first_row = recommended_dataframe.first()
top_users_like_count = 10

users_who_like_selected_movie = first_row["User_ID"]
remaining_user_count = top_users_like_count - len(users_who_like_selected_movie)

if remaining_user_count > 0:
    rows_to_consider = recommended_dataframe.take(remaining_user_count)[1:]
    for row in rows_to_consider:
        users_who_like_selected_movie.extend(row["User_ID"])
        remaining_user_count = top_users_like_count - len(users_who_like_selected_movie)
        if remaining_user_count <= 0:
            break

if len(users_who_like_selected_movie) > top_users_like_count:
    users_who_like_selected_movie = users_who_like_selected_movie[:top_users_like_count]

print(f"Recommended top 10 users for the movie {movie_id}: ", users_who_like_selected_movie)
```

Recommended top 10 users for the movie 1843: [10374, 5618, 11169, 7133, 1442, 14210, 2455, 3101, 4477, 265]

The first row of the DataFrame `recommended_dataframe` is retrieved and stored in `first_row`. The desired number of top users, `top_users_like_count`, is set to 10. The user IDs of users who liked the selected movie are extracted from `first_row` and stored in `users_who_like_selected_movie`. The remaining number of user IDs needed to reach `top_users_like_count` is calculated. If there are still more user IDs needed, additional rows from `recommended_dataframe` are considered. The user IDs from these additional rows are appended to `users_who_like_selected_movie`. The remaining user count is updated based on the new number of user IDs. If the desired count of `top_users_like_count` is reached or exceeded, the loop is terminated. If the length of `users_who_like_selected_movie` exceeds `top_users_like_count`, it is truncated to contain only the top `top_users_like_count` user IDs. Finally, the recommended top users for the selected movie are printed.

The troubles experienced

Issue 1- When we read dataset directly from .csv, we have to cast the columns for further usings. In order to prevent this situation, instead of read dataset directly from .csv, we used schema that contains string,int,nullable,long type indicators.

Dataset Issue- We've experienced that in first dataset that we were about to choose, `book_id`'s were string, but ALS function expects int values for item id's. So tried to convert this strings to int with unique values. But we couldn't

achieve the goal. ALS learning couldn't achieve significant success (because of duplicated datas).

Dataset2 Issue- Then we choose anime dataset, it had 50 million rows. It took 3 days and there was still no output. Then we realized that this kind of datasets should have been run in distributed async different devices.

Dataset3 Issue- Then we choose movie dataset, which contains movies.csv and users.csv. Then we inner joined them into one pyspark dataframe. But it had over 12 million rows so we trimmed it down. At the end we got one dataset with 100k rows.

Plotting Issue- We had to filter movie years like this ">=1968 year" to achieving better visualization of movie data. Because, before year 1968, there wasn't enough valueable data.

ALS Issue- While we running our ALS models, we've encountered an error which is `java.lang.StackOverflowError` error. We fixed it via configuration of spark session: `.config('spark.driver.extraJavaOptions', '-Xss3g')`. With this code we increased our RAM resource to 3 GB of RAM.

Another issue that needs clarification is to make the use of best_model from static to dynamic

Under normal circumstances, we had to manually type the name of the best model in the cosine similarity section. This caused our code to be unsustainable and not dynamic.

```
def add_model_results(result_dict, model_name, error_results):  
    """  
    Verilen model adı ve hata sonuçlarına göre "result_dict" sözlüğüne sonuçları ekleyen fonksiyon.  
    :param result_dict: Sonuçların ekleneceği sözlük.  
    :param model_name: Modelin adı.  
    :param error_results: Modelin hata sonuçları sözlüğü.  
    :return: None  
    """  
    result_dict[model_name] = {'RMSE': error_results['RMSE'], 'MSE': error_results['MSE']}  
    result_dict = {}
```

By applying these operations to each model, the results are written into the dictionary named result_dict with the model name as key and the error_results of the model as value.

```

model_listesi = ['model_1_1', 'model_1_2', 'model_1_3', 'model_2_1', 'model_2_2', 'model_2_3', 'model_3_1', 'model_3_2', 'model_3_3']

best_rmse = float('inf')
best_mse = float('inf')
best_result_index = None

# Her bir model için döngü
for i, model in enumerate(model_listesi):
    result = result_dict[model]
    rmse = result['RMSE']
    mse = result['MSE']

    if rmse <= best_rmse and mse <= best_mse:
        best_rmse = rmse
        best_mse = mse
        best_result_index = i

best_model = model_listesi[best_result_index]

print('Best model based on RMSE and MSE:', result_dict[best_model])
print(f'Best model is: {best_model}', "\n")
print("The best model is selected based on having the lowest RMSE or MSE, as these indicate the model's ability to minimize prediction errors. Lower values suggest that the model's predictions are closer to the actual values, resulting in better overall performance.")

```

Best model based on RMSE and MSE: {'RMSE': 0.8832646119053242, 'MSE': 0.7801563746442631}

Best model is: model_3_2_2

The best model is selected based on having the lowest RMSE or MSE, as these indicate the model's ability to minimize prediction errors. Lower values suggest that the model's predictions are closer to the actual values, resulting in better overall performance.

`best_result_index` represents the index number of the best value found in `model_list`. Then we assign which model it is to `best_model` with its name. Then we use the `result_dict[best_model]` structure to see the RMSE and MSE values of the `best_model`. `best_model` is the key, that is, the name of the model in the `result_dict`, and value is the RMSE and MSE values of the model whose name is mentioned in this dict.

Let's not forget that the type of `best_model` is string when we come this far, that is, we have to manually write the name of the model in the `cosine_similarity` function. However, since we think that this is contrary to writing sustainable code, we resorted to this way.

```

# globals() kullanarak dictionary'yi çağırma
if best_model in globals():
    my_dict = globals()[best_model]

```

The `if best_model in globals():` statement checks whether the variable named `best_model` exists in the `globals()` dictionary. If the name `best_model` exists in the dictionary, the following block of code is executed.

The expression `my_dict = globals()[best_model]` takes a value using the key named `best_model` of the `globals()` dictionary and assigns it to the variable named `my_dict`. This value represents the value associated with the name `best_model`.

For example, if the name `best_model` is defined as `model1` and a variable named `model1` is defined, the expression `my_dict = globals()[best_model]` assigns the value of variable named `model1` to variable named `my_dict`. So `my_dict` contains the dictionary represented by the `model1` variable.

we can see or code is made dynamic:

```

from pyspark.sql import Row
from numpy import linalg as LA
from pyspark.sql import functions as F
import numpy as np

def cosine_similarity(vector_1, vector_2):
    v1 = np.asarray(vector_1)
    v2 = np.asarray(vector_2)
    cs = v1.dot(v2) / (LA.norm(v1) * LA.norm(v2))
    return cs

def compute_similarity(itemFactors, movie_id):
    item = itemFactors.where(itemFactors.id == movie_id).select('features')
    item_features = item.rdd.map(lambda x: x.features).first()
    temp = []
    for row in itemFactors.rdd.toLocalIterator():
        if row.id != movie_id:
            similarity_score = cosine_similarity(row.features, item_features)
            temp.append((row.id, similarity_score))
    R = Row('Movie_ID', 'Similarity_Score')
    similar_items_df = spark.createDataFrame([R(col[0], float(col[1])) for col in temp])
    return similar_items_df

item_factors = my_dict['model'].itemFactors
similar_items_df = compute_similarity(item_factors, 1843) # Replace X with the actual movie ID

# Aggregate the similarity scores using reduceByKey()
# Aggregation is a process in data analysis where data is grouped together and summarized to provide meaningful insights.
similarity_sum_rdd = similar_items_df.rdd.map(lambda x: (x['Movie_ID'], x['Similarity_Score'])).reduceByKey(lambda a, b: a + b)

# Create a new DataFrame from the aggregated RDD
R = Row('Movie_ID', 'Similarity_Sum')
similarity_sum_df = spark.createDataFrame([R(movie_id, similarity_sum) for movie_id, similarity_sum in similarity_sum_rdd.collect()])

# Sort the DataFrame by Similarity_Sum in descending order
sim_res = similarity_sum_df.orderBy(F.col('Similarity_Sum').desc()).limit(10)
sim_res.show()

```

References

We've used kaggle for both datasets and for looking how users implement their codes.

- <https://www.kaggle.com/code/prashant111/recommender-systems-in-python/input?select=ratings.csv>
- https://www.kaggle.com/datasets/hernan4444/anime-recommendation-database-2020?select=rating_complete.csv
- https://www.kaggle.com/datasets/aprabowo/indonesia-tourism-destination?select=tourism_rating.csv
- <https://www.kaggle.com/code/sherinclaudia/movie-rating-prediction/notebook>
- <https://www.kaggle.com/datasets/rishitjavia/netflix-movie-rating-dataset>
- <https://www.kaggle.com/datasets/ruchi798/bookcrossing-dataset>
- <https://www.kaggle.com/datasets/samlearner/letterboxd-movie-ratings-data>
- <https://www.kaggle.com/datasets/CooperUnion/anime-recommendations-database>
- <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>
- https://seaborn.pydata.org/examples/palette_choices.html
- <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html>
- <https://www.askpython.com/python/examples/rmse-root-mean-square-error>